

DRAVYANTRA PLATFORM GUIDE

Complete Reference Manual: Technology Stack, Business Logic & Calculations, and Dashboard Operations

Document Type: System & Operations Manual

Target Audience: Management, Developers, Fleet Operators

Format: Executive Summary

1. Technology Stack & Why We Use It

DravYantra is an IoT-powered fleet tracking and fuel intelligence platform. Each technology was selected to guarantee real-time speed, accuracy for financial calculations, and user-friendly mobile and web interfaces.

Application Component	Technology Used	Why We Selected This Tech Stack (Simple Reasons)
Fleet Owner Mobile App iOS & Android	Flutter (Dart) Firebase Auth & Push Notifications	• Single Codebase: Works smoothly on both Android phones and iPhones without building two separate apps. • Fast & Responsive: Smooth 60 FPS performance for live vehicle maps and fuel dials. • Instant Alerts: Firebase delivers instant push notifications directly to the owner's phone when fuel theft or overspeeding happens.
Fleet Manager Web Dashboard Web Portal	Next.js (React) TypeScript & Tailwind CSS	• Fast Loading: Next.js loads web pages instantly so fleet managers can track dozens of trucks simultaneously. • Error Prevention: TypeScript ensures all trip data and financial calculations are strictly verified before display. • Clean Layouts: Tailwind CSS creates modern, responsive dashboards that look sharp on laptops, desktops, and tablets.
Super-Admin Platform Console Admin Portal	Flutter Web & Desktop Material Design 3	• High-Density Management: Built for master administrators to manage hundreds of client organizations, verify business accounts, and manage GPS devices. • Unified Desktop Experience: Provides a desktop-style interface with data tables and audit logs.
Backend API & Telemetry Engine Core Engine	Node.js & Express.js	• Handles High Traffic: IoT GPS devices send location and fuel readings every 5 seconds. Node.js processes thousands of live pings simultaneously without slowing down. • Modular Design: Separates GPS math, fuel checks, and safety alerts into clean, reliable background engines.
Database & Storage ACID Relational	PostgreSQL 15+ Database Triggers & Pooling	• Financial Accuracy: Every rupee saved or lost is calculated accurately using ACID transactions. • Built-in Safety Triggers: If any calculation is missed, the database automatically recalculates fuel usage and costs on the spot.

2. Backend Calculations & Business Logic (In Simple Human Terms)

Commercial trucks waste massive amounts of money on fuel due to rash driving, excessive idling, and fuel theft. DravYantra takes raw GPS and fuel sensor readings and converts them into clear financial and environmental insights.

Feature 1: Speed-to-Mileage Calculation (How Driving Speed Affects Mileage)

The Problem: Heavy trucks burn much more fuel when driven too fast because of wind resistance. A truck cruising at 50 km/h gets great mileage, while the same truck at 100 km/h wastes fuel.

How the Backend Solves It: The backend checks the truck's current speed and dynamically assigns the correct mileage efficiency based on standard commercial vehicle standards:

40 to 59 km/h: 5.0 km/L (Best Eco Mileage)

60 to 69 km/h: 4.0 km/L (Normal Baseline)

80 km/h: 3.4 km/L

100 km/h: 3.0 km/L

120+ km/h: 2.6 km/L

Formula: Fuel Used (Liters) = Trip Distance (km) ÷ Current Mileage (km/L)

Real Example: A truck travels a 400 km trip:

- If driven smoothly at **50 km/h** (5.0 km/L) → Fuel Used = $400 \div 5.0 = 80$ Liters.
- If driven fast at **100 km/h** (3.0 km/L) → Fuel Used = $400 \div 3.0 = 133.3$ Liters (Wastes 53.3 Liters!).

Feature 2: Fuel Saved & Money Saved (Eco-Driving Profit)

The Problem: Fleet owners want to know the exact rupee amount saved when drivers drive in the safe eco-speed zone (40–59 km/h).

How the Backend Solves It: The backend compares actual fuel used against standard baseline fuel (at 4.0 km/L) and multiplies the saved liters by the local fuel price (e.g., ₹100 per liter from Fleet Settings).

1. Expected Baseline Fuel = Distance ÷ 4.0 km/L

2. Fuel Saved (Liters) = Expected Fuel – Actual Fuel Used

3. Money Saved (₹) = Fuel Saved × Fuel Price per Liter (₹100)

Real Example: A truck covers 400 km at 50 km/h (actual fuel used = 80 Liters):

- Baseline Expected Fuel = $400 \div 4.0 = 100$ Liters.
- Fuel Saved = 100 Liters – 80 Liters = **20 Liters Saved.**
- Money Saved = 20 Liters × ₹100 = **₹2,000 Direct Cash Saved on this trip.**

Feature 3: Speeding Fuel Wasted & Financial Loss

The Problem: Drivers often overspeed on highways to finish trips quickly, burning excess fuel and increasing accident risk.

How the Backend Solves It: The backend calculates how much extra fuel was burned compared to normal driving and translates it into direct financial loss.

1. Speeding Fuel Wasted = Actual Fuel Used – Expected Baseline Fuel

2. Money Lost (₹) = Speeding Fuel Wasted × Fuel Price (₹100)

Real Example: A truck covers 400 km at 120 km/h (2.6 km/L efficiency):

- Actual Fuel Used = $400 \div 2.6 = 153.85$ Liters (vs 100 Liters baseline).
- Fuel Wasted = 153.85 – 100 = **53.85 Liters Wasted.**
- Money Lost = 53.85 Liters × ₹100 = **₹5,385.00 Lost due to speeding.**

Feature 4: Engine Idling Time & Money Wasted (Realistic Commercial Stats)

The Problem: Drivers often leave the engine running while parked at tolls, loading docks, or tea stalls, burning fuel needlessly.

How the Backend Solves It: When the vehicle has Engine ON but Speed is 0 km/h, the backend tracks idle seconds. In real-world stats, a heavy commercial truck in idle state wastes roughly **0.03 to 0.07 Liters of fuel per minute** (averaging ~0.05 Liters per minute / 0.00083 Liters per second). When converting to money at a fuel price of **₹100 per Liter** (fetched from Fleet Settings), this equals **₹0.08 wasted per second** (around ₹4.80 to ₹5.00 per minute).

1. Idling Fuel Wasted (Liters) = Idling Time in Seconds × 0.00083 Liters/second (~0.05 L/minute)

2. Idling Money Wasted (₹) = Idling Time in Seconds × ₹0.08 per second (at ₹100/L fuel price)

3. Automatic Alerts: Warning alert sent at 5 minutes (300s), Critical alert sent at 15 minutes (900s).

Real Example: A driver leaves the engine running for 20 minutes (1,200 seconds) while loading:

- Idling Fuel Wasted = 1,200 seconds × 0.00083 = **1.0 Liter of fuel wasted.**
- Idling Money Wasted = 1,200 seconds × ₹0.08 = **₹96.00 Wasted.**

Feature 5: Real-Time Fuel Theft Detection (Continuous 5-Second Cloud Comparison)

How the Hardware & Cloud Work: When a vehicle engine is turned OFF, the IoT device does not send data continuously. Instead, for every 5 seconds during active operation, OR whenever the vehicle stops and starts again after some time, the device sends its latest fuel reading to the cloud.

How the Backend Detects Theft: On every incoming packet, the backend performs a continuous comparison between the **new fuel reading** and the **previous recorded fuel reading**. Normal 5-second driving consumption is tiny (under 0.05 Liters). If there is a sudden drop of **greater than 3 Liters (Drop ≥ 3.0 L)** between readings (whether during transit or after waking up from a stop), the backend immediately flags it as a **Fuel Theft Incident** with the exact location!

Theft Condition: (Previous Fuel Reading – New Fuel Reading) ≥ 3.0 Liters.

Theft Money Loss (₹) = Liters Dropped $\times$ Fuel Price per Liter (₹100 from Fleet Settings)

Real Example: A truck was parked with **180 Liters**. When the vehicle starts in the morning or sends a reading, the fuel is **145 Liters**:

- Fuel Drop = $180 - 145 = 35$ Liters Drop (greater than 3 Liters threshold).
- Financial Loss = $35 \text{ Liters} \times ₹100 = ₹3,500.00$ Stolen.
- Backend Action: Immediately triggers an emergency Critical Fuel Theft Alert with GPS location to the owner's phone.

Feature 6: Carbon (CO2) Emissions, CO2 Avoided & Carbon (C) Reduction

The Problem: Fleet companies must track and report environmental pollution (ESG reporting) to corporate clients.

How the Backend Solves It: Burning 1 Liter of diesel produces exactly **2.68 kg of Carbon Dioxide (CO2)**. When fuel is saved, CO2 is avoided. The pure elemental carbon mass is calculated using the molecular ratio of Carbon to CO2 ($12 \div 44 = 0.2727$).

1. Gross CO2 Emitted (kg) = Fuel Used (Liters) $\times$ 2.68 kg CO2

2. CO2 Avoided (kg) = Fuel Saved (Liters) $\times$ 2.68 kg CO2

3. Pure Carbon Reduced (kg) = CO2 Avoided $\times$ (12 $\div$ 44)

Real Example: A fleet saves **1,500 Liters of diesel** in a month across all its trucks:

- CO2 Avoided = $1,500 \text{ Liters} \times 2.68 = 4,020$ kg of CO2 Saved (4.02 Metric Tonnes).
- Pure Carbon Mass Reduced = $4,020 \times (12 \div 44) = 1,096.36$ kg of Pure Carbon Reduced.

Feature 7: Driver Safety Scorecard (Fair 0 to 100 Rating)

The Problem: Long-distance drivers naturally encounter more road events than short-distance drivers. Scoring must be fair.

How the Backend Solves It: Every driver starts with **100 points**. Penalties are normalized per **100 km of driving** across four categories:

- **Overspeeding Penalty:** (Speed Events $\div$ Distance) $\times 100 \times 5.0$ (Max -25 pts)
- **Excess Idling Penalty:** (Idle Hours $\div$ Distance) $\times 100 \times 4.0$ (Max -25 pts)
- **Fuel Inefficiency Penalty:** If mileage is lower than target (Max -25 pts)
- **Harsh Braking / Rash Driving:** (Harsh Braking Events $\div$ Distance) $\times 100 \times 5.0$ (Max -25 pts)

Final Driver Score = 100 – (Overspeed + Idling + Inefficiency + Rash Driving Penalties)

Score Ratings: 90–100: Excellent (Bonus Eligible) | 75–89: Good | 60–74: Needs Training | <60: High Risk.

3. DravYantra Fleet Manager Dashboard Features

The Fleet Manager Dashboard is the central web control center for fleet managers to monitor daily logistics operations, fuel efficiency, and driver safety.

Dashboard Feature / Page	What Fleet Managers Can Do (Functions & Capabilities)
1. Executive Overview	View high-level summary cards showing active trucks running, idling, or parked; total kilometers driven today; total fuel cost; net money saved; and monthly fuel vs loss trend charts.
2. Live Fleet Tracking Map	Interactive full-screen map showing all vehicles in real-time. Hover over any truck to see live speed, fuel percentage, driver name, engine status, and geofenced warehouse boundaries.
3. Trips Management	Create and dispatch new trips with origin, destination, assigned driver, and vehicle plate. Track active trip progress, view historical route playback on a map, and review trip profitability.
4. Fuel Intelligence & Audits	Monitor real-time fuel levels, view an audit log of all fuel refills (to cross-check fuel pump receipts), and review fuel theft incidents with timestamps and GPS links.
5. Audited Savings Wallet	Review audited fleet savings broken down into three clean categories: (1) Verified Net Savings from eco-driving, (2) Losses Prevented by real-time alerts, and (3) Identified Waste (idling + speeding).
6. Carbon & ESG Reporting	Track corporate sustainability numbers: total CO2 emitted, CO2 avoided, and carbon reduced. Export ready-to-share ESG compliance reports for corporate customers with a single click.
7. Driver Scorecards & Safety	View fleet driver rankings from best to worst. Inspect individual driver profiles, see specific penalty breakdowns (harsh braking, overspeeding, idling), and provide targeted driver coaching.
8. Vehicle Health & Maintenance	Check mechanical health scores (0–100) for all trucks, monitor chassis vibration readings, track mileage degradation, and set reminders for oil changes, tire rotations, and fitness certificates.
9. Alerts & Incident Watchtower	Live feed of all alerts categorized by Critical (Theft, Tampering), Warning (Overspeed, Long Idling), and Info (Refills). Managers can acknowledge, investigate, and mark incidents as resolved.
10. Reports & Exports Hub	Generate and download custom date-range reports in PDF, Excel (.xlsx), and CSV formats for trips, fuel consumption, driver compliance, and executive ROI summaries.
11. Fleet Settings	Customize fleet-wide parameters such as speed limit thresholds (e.g. 70 km/h vs 80 km/h), fuel theft sensitivity, local diesel price per liter (e.g. ₹100/L), and notification routing.
12. Helpdesk & Support	Raise support tickets directly to the DravYantra technical engineering team and track resolution progress and SLA updates in real time.

4. DravYantra Super-Admin Dashboard Features

The Super-Admin Dashboard is the master platform operations console used by DravYantra administrators to manage client organizations, hardware inventory, and system security.

Admin Feature / Section	Admin Responsibilities & How It Works
1. Platform Operations Center	Monitors overall system health, including real-time telemetry ingestion speed (packets per second), database performance, active client tenants, and total connected vehicles.
2. Organization Onboarding	Reviews new fleet business sign-ups, verifies uploaded company registration documents and tax IDs, approves or rejects applications, and sets vehicle fleet size limits.
3. IoT Device Inventory	Registers and manages GPS trackers and OBD-II hardware by IMEI number. Links devices to client vehicles, monitors SIM card connectivity status (Online, Offline, Weak Signal), and manages firmware updates.
4. Global Vehicle Registry	Search and inspect any vehicle registered across the entire platform. Admin can check hardware assignment history and unlink devices from decommissioned trucks.
5. Global Trip Surveillance	Platform-wide view of all active trips across all organizations. Allows administrators to investigate anomalies and close or reset stalled trips.
6. Subscriptions & Billing	Configure subscription tiers (Starter, Growth, Enterprise), manage monthly and annual billing cycles, generate tax invoices, and track payment receipts.
7. Global User & Role Management	Manage user accounts across all roles (Super Admin, Support Engineer, Fleet Owner, Dispatcher, Driver). Enforce security policies, password resets, and Multi-Factor Authentication.
8. Security Watchtower	Monitors platform-wide security threats, detecting multi-vehicle fuel theft rings across regions and tracking hardware tampering or power disconnection events.
9. Audit Logs & Compliance	Immutable security log recording every administrative action (who approved an account, changed a role, or modified settings) with timestamps and IP addresses for full legal compliance.
10. Global Support Desk	Consolidated queue of all customer support tickets raised by fleet managers. Assigns tickets to support engineers, manages resolution SLAs, and maintains customer communication.
11. Recycle Bin & Soft Deletion	Provides a 30-day safety recovery period for deleted vehicles or organizations. Deleted items can be restored with a single click before permanent automated database purging.

5. Quick Reference Summary Table

Metric / Calculation	Simple Plain English Formula	Key Default Setting
Distance Traveled	Earth Surface Distance between GPS Points (Filters jumps > 5 km)	Earth Radius = 6,371 km
Dynamic Mileage	Speed 40–59 km/h: 5.0 km/L 60–69 km/h: 4.0 km/L Higher speed reduces mileage	Baseline = 4.0 km/L
Fuel Used	Trip Distance (km) ÷ Current Mileage (km/L)	Dynamic based on speed
Money Saved	(Expected Fuel – Actual Fuel Used) × Fuel Price	Fuel Price = ₹100 / Liter (from Fleet Settings)
Speeding Loss	(Actual Fuel Used – Expected Fuel) × Fuel Price	Applies when Speed > 69 km/h
Idling Loss	Idling Time in Seconds × ₹0.08 per second (~0.05 L/minute wasted)	₹0.08 / second (at ₹100/L fuel price)
Fuel Theft Detection	Continuous 5s / restart check: Previous Fuel – Current Fuel ≥ 3.0 Liters	Threshold = 3.0 Liters drop
CO2 Emitted	Fuel Used (Liters) × 2.68 kg CO2	2.68 kg CO2 / Liter
CO2 Avoided / Saved	Fuel Saved (Liters) × 2.68 kg CO2	2.68 kg CO2 / Liter
Carbon (C) Reduced	CO2 Avoided × (12 ÷ 44)	Ratio = 0.2727
Driver Score	100 – (Overspeeding + Idling + Inefficiency + Rash Driving Penalties per 100 km)	0 to 100 Points Scale